

Die Java-Codebeispiele liegen in Form eines Eclipse-Projekts vor. Wie sie dieses nutzen können, ist in der Readme-Datei beschrieben.

Die Beispiele sind thematisch auf Pakete verteilt. Bitte beachten Sie, dass dies *keine* sinnvolle Paketstruktur für ernsthaftes Programmieren darstellt.

Bei einigen Codebeispielen finden Sie die Erläuterungen auch als Kommentar direkt im Quelltext.

package bsp_01 _deklarationstyp

1. Subtyppolymorphie in ihrer primitivsten Form:

Zeile 5 zeigt, dass man einer Referenz-Variablen nicht nur eine Referenz zuweisen kann, die vom selben Typ ist wie die Variable, sondern auch Referenzen die als statischen Typ einen Subtyp des statischen Typs der Variablen haben. Im Beispiel steht rechts des Zuweisungsoperators das Ergebnis eines Konstruktoraufrufs. Ein solcher Ausdruck hat ja bekanntlich als deklarierten Rückgabetyt immer den Typ der Klasse, von der er ein Exemplar erzeugt.

2. Der Java-Compiler interessiert sich nur für statische Typen.

Der Compilerfehler in Zeile 6 entsteht, weil versucht wird, auf einem Ausdruck vom statischen Typ Super eine Methode aufzurufen, die für diesen Typ nicht definiert ist („The method m() is undefined for the type Super“).

Package bsp_02 _casting

Hier geht es um Casts. Bekanntlich ist ein Cast eine Zusicherung an den Compiler, dass das von einer Referenz referenzierte Objekt zur Laufzeit einen ganz bestimmten Typ haben wird. So eine Zusicherung kann vom Compiler naturgemäß so gut wie nicht überprüft werden, weswegen jeder notwendige Cast auch mit dem Verlust der Typsicherheit einhergeht.

In Zeile 7 haben wir zunächst einmal keinen Cast, sondern den bekannten Fall, dass der Compiler uns auf einem Ausdruck nur Methodenaufrufe / Attributzugriffe erlaubt, die für den statischen Typ des Aufrufs auch definiert sind. Und für Object – der statische Typ von o1 – ist nun einmal kein Attribut name deklariert.

In Zeile 9 haben wir nun einen Cast, d.h. wir sichern dem Compiler zu, dass o1 zur Laufzeit ein Person-Exemplar referenzieren wird. Das glaubt er uns. Denn das, was in Zeile 6 auf der rechten Seite steht, interessiert ihn in Zeile 9 nicht, er kennt nur den statischen Typ von o1, und eine Variable vom Typ Object kann ja zur Laufzeit tatsächlich ein Person-Exemplar enthalten, denn Person ist Subtyp von Object. Und da der Compiler den Cast akzeptiert, erlaubt er uns auch den Attributzugriff in Zeile 10, denn eine Person hat ein Attribut name.

Interessant ist nun die Frage, warum der Compiler für den fast genauso gelagerten Fall in Zeile 13 den Cast *nicht* akzeptiert. Nun, der statische Typ von s ist String. Und daher kann s auch zur Laufzeit niemals eine Person referenzieren. Denn dazu müsste Person Subtyp von String sein. Nun weiß der Compiler zwar i.A. gerade *nicht*, welche Subtypen es von einem Typ irgendwann einmal geben könnte. Aber im Fall von String weiß er es, denn die Klasse String ist als final deklariert, und das heißt, dass von ihr keine Subtypen abgeleitet werden *können*.

Warum akzeptiert der Compiler aber den Cast in Zeile 16? Schließlich sichern wir ihm zu, dass p3 zur Laufzeit ein Objekt referenzieren wird, welches das Interface Printable implementiert. Die Variable p3 ist aber vom statischen Typ Person und die Klasse Person implementiert Printable nicht, was ja auch der Compiler sehen kann. Richtig... aber p3 könnte zur Laufzeit ein Exemplar eines Subtyps von Person referenzieren, nämlich ein Exemplar einer Subklasse, die ihrerseits Printable implementiert! Wir könnten z.B. eine Klasse Student schreiben, mit „Student extends Person implements Printable“. Und ob das der Fall sein wird, kann der Compiler nicht entscheiden, weswegen er genötigt ist, uns zu glauben.

Dass wir gelogen haben, kann erst zur Laufzeit erkannt werden. Wenn wir die Zeilen 6 – 13 auskommentieren, also nur die Zeilen 15 – 17 in main() bleiben, können wir das Programm kompilieren und ausführen. Es kommt dann zu einer ClassCastException in Zeile 16.

Kommen wir zu Zeile 19. Auf der rechten Seite der Zuweisung haben wir einen Konstruktoraufruf. Dessen statischer Typ ist definitionsgemäß der Typ der Klasse, deren Konstruktor aufgerufen wird, also Object. Der Compiler akzeptiert den Cast auf Person, denn er ist prinzipiell möglich: Eine Referenz vom statischen Typ Object *könnte* ja auch eine Person referenzieren. Dass das hier mit Sicherheit nicht der Fall ist, kann der Compiler nicht feststellen, denn er vergleicht ja nur statische Typen.

Zur Laufzeit würde (wenn wir den Rest so auskommentieren, dass keine Compilerfehler mehr auftreten und das Programm bis zu dieser Stelle läuft) auch Zeile 19 zu einer ClassCastException führen. Denn die VM stellt fest, dass das erzeugte Objekt eben doch keine Person *ist*. Hier haben wir übrigens den interessanten Fall, dass auch ein Cast, der gar nicht nötig gewesen wäre, zu einem Laufzeitfehler führen kann. Ohne den Cast wäre Zeile 19 ja völlig unproblematisch, sowohl für den Compiler als auch zur Laufzeit. Gute IDEs warnen einen vor solchen unnötigen Casts. Solche Warnungen weisen zwar nicht auf Compilerfehler hin (sonst wären sie nicht nur Warnungen), aber sie sind meist ein Anzeichen für logische Fehler oder Missverständnisse des Programmierers.

Package bsp_03_dynamische_bindung

Die Variable s ist vom statischen Typ Super. Die einzige zum Aufruf passende Methode in Super ist die parameterlose Methode m(). Der Compiler schreibt also einen Aufruf der Methode mit eben dieser Signatur in den Bytecode. Zur Laufzeit stellt die Laufzeitumgebung (VM) fest, dass s ein Objekt vom Typ Sub referenziert. Aufgrund der dynamischen Methodenwahl (dynamisches Binden) sucht die Laufzeitumgebung nun in Sub nach der vom Compiler ausgewählten Methodensignatur. Da im Typ Sub die von Super

geerbte Methode `m()` überschrieben wurde, findet die VM die überschreibende Methode `m()` und führt diese aus. Die Ausgabe ist also:

Methode `m` von `Sub`

Package `bsp_04_statische_bindung_bei_static`

Fast das Gleiche wie im vorigen Beispiel, nur sind die Methoden `m()` in `Super` und `Sub` diesmal als Klassenmethoden (`static`) deklariert! Aber auch hier werden die Methoden `m()` *scheinbar* auf Objekten aufgerufen. In Wirklichkeit haben statische Methoden aber kein Empfängerobjekt, sie werden über den Klassennamen angesprochen. Leider erlaubt Java die irreführende Art des Aufrufs, die im Beispiel gezeigt wird. In Wirklichkeit wandelt aber der Compiler die Aufrufe in die Form um, die man für statische Elemente ausschließlich verwenden sollte, nämlich in Aufrufe über den Klassennamen.

Eclipse weist uns auf die unangebrachten Aufrufe mit mehreren Warnings hin: „The static method `m()` from the type `Sub` should be accessed in a static way“. Mit Hilfe der QuickfixFunktion (`STRG + 1`) kann man Eclipse die Aufrufe umwandeln lassen. Das Ergebnis wäre für die Methode `main()` der Klasse `Test`:

```
Super sup1 = new Super();
Super.m();
Super sup2 = new Sub();
Super.m();
Sub.m();
```

womit die Objekterzeugungen in der ersten und dritten Zeile für die Methodenaufrufe völlig irrelevant sind. Das waren sie natürlich vorher auch schon, aber jetzt sieht man es sofort. Die Ausgabe des Programms lautet folgerichtig (auch in der Originalversion):

Methode `m` von `Super`
Methode `m` von `Super`
Methode `m` von `Sub`

Wenn Ihnen die Ausgabe bzgl. der Zeile `((Sub) s2).m();` rätselhaft erscheint, beachten Sie bitte: Der statische Typ eines „gecasteten Ausdrucks“ ist definitionsgemäß der Typ, auf den gecastet wird. Hier erfolgt der Aufruf auf dem Ausdruck `(Sub) s2`. Dessen statischer Typ ist `Sub`.

Package `bsp_05_statische_bindung_von_attributen`

Attributzugriffe werden in Java statisch gebunden (also durch den Compiler), und zwar nicht nur Klassenattribute (`static`), sondern auch „normale“ Attribute. Deshalb ist bzgl. der Auflösung des Attributzugriffs in Zeile 6 auch nur der statische Typ von `s` relevant und es wird das Attribut `a` der Klasse `Super` ausgegeben.

Zeile 7 zeigt einen ganz normalen dynamisch gebundenen Methodenaufruf: Zur Laufzeit wird die Methode `m1()` von `Sub` gewählt und da diese im Code der Klasse `Sub` definiert ist, *muss* sie auf das Attribut `a` von `Sub` zugreifen!

Zeile 8 zeigt, dass in der Klasse Sub das geerbte Attribut a aus Super zwar durch das gleichnamige Attribut von Sub *verdeckt* wurde, aber trotzdem nach wie vor vorhanden ist: Ein Sub hat ja auch die Methoden aus Super geerbt und kann sie aufrufen. Genau das tut die Methode m2() in Zeile 32 mit der geerbten Methode x(), Und da x() eine Methode ist, die im Code der Klasse Super definiert ist, kann *ihr* Attributzugriff sich nur auf das Attribut aus Super beziehen. Das in Zeile 8 erzeugte Sub-Exemplar hat also offenbar zwei Attribute a, von denen eines verdeckt/versteckt, aber über geerbte Methoden dennoch zugreifbar ist. Damit man nicht aus Versehen Attribute von Superklassen verdeckt, warnt uns Eclipse bzgl. der Deklaration in Zeile 25: „The field Sub.a is hiding a field from type Super.“

Package bsp_06_konstrukturen_1

Zeile 4: Die Klasse Lebewesen hat keinen explizit angegebenen Konstruktor. Dennoch wird hier ein Exemplar erzeugt. Erklärung: Hat eine Klasse keinen explizit deklarierten Konstruktor, bekommt sie automatisch einen parameterlosen Default-Konstruktor, dieser wird hier verwendet.

Zeile 5: Die Klasse Person *hat* einen explizit deklarierten Konstruktor. Dadurch entfällt der implizite Default-Konstruktor. → Compilerfehler „The constructor Person() is undefined“

Zeilen 22, 32, 40: In den Klassen Person, Student, Professor werden für das zu initialisierende Attribut die gleichen Bezeichner verwendet wie für den Parameter, der zur Initialisierung verwendet wird. Das ist bei Konstrukturen gängige Praxis. Ein Parameter ist jedoch bzgl. seiner Methode bzw. hier seines Konstruktors eine lokale Variable und verdeckt das gleichnamige Attribut. Durch Voranstellen von this wird jeweils klargestellt, dass auf der linken Seite der Zuweisung die Variable des aktuellen Objekts gemeint ist, also das Attribut.

Zeile 31: In der Klasse Student verwendet der Konstruktor in der ersten Zeile explizit einen Konstruktor der Superklasse.

Zeile 39: Jeder Konstruktor, der *nicht* explizit in der ersten Zeile einen anderen Konstruktor der gleichen Klasse oder einen Konstruktor der Superklasse aufruft, ruft implizit immer den parameterlosen Konstruktor der Superklasse auf. Da Person aber keinen parameterlosen Konstruktor hat, geht dies hier nicht und es kommt zu einem Compilerfehler „Implicit super constructor Person() is undefined. Must explicitly invoke another constructor“. Zur Beseitigung des Fehlers würde man analog zu Student vorgehen.

Package bsp_07_konstrukturen_2

Dieses Beispiel demonstriert ein Problem, das entstehen kann, wenn aus einem Konstruktor Methoden aufgerufen werden und eine Subklasse diese überschreiben darf. Erläuterung: Der Superklassenkonstruktor wird immer vor Rumpf des Subklassenkonstruktors ausgeführt. Der Superklassenkonstruktor ruft die Methode m() auf. Diese wurde in der Subklasse überschrieben. Aufgrund dynamischer Bindung – das aktuelle Objekt ist das, welches gerade initialisiert wird, und das ist ja vom Typ Sub – wird die Methode m() der Subklasse gewählt. Diese will auf dem Subklassenattribut eine Methode aufrufen, aber das Subklassenattribut ist noch gar nicht initialisiert. → NullPointerException.

Package bsp_08_abstrakte_klassen

Das Beispiel zeigt, wie man aus einer konkreten Methode einer abstrakten Klasse abstrakte Methoden der konkreten Subklassen aufrufen kann (Zeile 12), obwohl ja völlig unbekannt ist, wie deren Implementierung in diesen Subklassen aussehen wird. Dass eine solche Implementierung in jeder konkreten Subklasse vorhanden sein wird, wird eben durch die Deklaration der abstrakten Methode in der abstrakten Klasse (Zeile 17) erzwungen und vom Compiler überprüft.

Package bsp_09_polymorphie

Die Klassen dieses Packages zeigen die Verwendung und die Eigenschaften von Subtyppolymorphie, parametrischer Polymorphie und beschränkter parametrischer Polymorphie. Um das Beispiel möglichst einfach zu halten wurde ein minimaler Behälter verwendet, der genau ein Element aufnehmen kann.

- Klasse `A_SubtypPolymorphie`: Es wird `Object` als Elementtyp verwendet, man kann also beliebige Objekte in den Container füllen, bekommt diese aber auch nur als `Object` zurück → Cast nötig, Verlust an Typsicherheit. In Java vor Java 5 der Normalfall.
- Klasse `B_ParametrischePolymorphie`: Löst das vorgenannte Problem: Der Verwender des Containers legt den Elementtyp bei dessen Erzeugung fest, die Notwendigkeit des Casts entfällt. Der Erbauer des Containers weiß nichts darüber, welchen Elementtyp der Verwender wählen wird.
- Klasse `C1_BeschraenkteParametrischePolymorphie_Versuch1`: Ein Versuch, innerhalb des Containers einen Vergleich mittels `compareTo()` anzustellen, scheitert, weil der Compiler keine Annahmen über den Elementtyp treffen kann und daher auf dem Element nur den Aufruf von Methoden erlaubt, die für `Object` definiert sind.
- Klasse `C2_BeschraenkteParametrischePolymorphie_Versuch2`: Der Programmierer des Containers hat als obere Schranke für den Typparameter den Typ `Comparable` angegeben. Damit kann der Verwender nur noch solche Elementtypen wählen, die Subtyp von `Comparable` sind. Vorteil: Der Compiler kennt die Schranke und kann daher innerhalb des Containers den Aufruf der Methode `compareTo()` auf dem Element erlauben. Das Beispiel ist in dieser Vereinfachung natürlich nicht wirklich sinnvoll. Ein komplexeres aber dafür praxisnahes Beispiel wäre ein binärer Suchbaum, bei dem Sie ja zur Einordnung der Element diese ebenfalls vergleichen müssen. Bei der Klasse `C2_BeschraenkteParametrischePolymorphie_Versuch2` erhalten wir zwei Warnungen, dass wir den Typ `Comparable` ohne Typparameter verwenden, obwohl dessen Deklaration einen Typparameter vorsieht. Das Problem, vor dem wir hier gewarnt werden, ist wieder ein Typproblem: Der Verwender könnte als Elementtyp einfach `Comparable` selbst nehmen, womit man auch Objekte verschiedener Klassen, die `Comparable` implementieren, in den Container füllen könnte. Typischerweise führt dies aber zu einer `ClassCastException` beim Vergleich mit `compareTo()`.
- Klasse `C3_BeschraenkteParametrischePolymorphie`: Die Warnungen wurden beseitigt, indem bei der Angabe der Schranke für den Typparameter des Containers dieser erneut verwendet wurde. Die dahinterstehende Logik ist, dass wir als

Schranke ja nicht einfach allgemeine Vergleichbarkeit haben wollen, sondern Vergleichbarkeit eben mit dem Elementtyp, den der Verwender wählt. Damit das funktioniert, muss der Elementtyp natürlich deklarieren, vergleichbar mit sich selbst zu sein. Genau das ist für die Typen, die Comparable implementieren, i.A. der Fall. So implementiert
z.B. der Typ String nicht einfach Comparable, sondern eben Comparable<String>.

Package bsp_10_visibility

Hier geht es um die verschiedenen Modifikatoren, mit denen sich die Sichtbarkeit von Klassen, Methoden, Attributen usw. beeinflussen lassen. In jeder der vier Klassen findet sich ein Attribut vom Typ A, sowie eine Methode test(), in der versucht wird, auf dem Attribut seine vier Methoden aufzurufen. die jeweils nach ihrer Sichtbarkeitsstufe benannt sind. Beachten Sie, dass Sichtbarkeitsfragen in Java durch den Compiler geklärt werden. Es ist also nicht nötig, ein komplettes Programm zu schreiben, dass irgendwie lauffähig ist und Ausgaben macht. Stattdessen reichen zur Demonstration der Sichtbarkeit eines Elements Zugriffsversuche und die Analyse der auftretenden Compilerfehler bzw. deren Nichtauftreten. Bemerkenswert sind dabei vor allem folgende Sachverhalte:

- Aus einem Exemplar der Klasse A sind private Methoden eines *anderen* Exemplars dieser Klasse sichtbar. Die Kapselung durch Sichtbarkeitsmodifikatoren findet als offensichtlich nicht auf der Exemplar- sondern auf der Klassenebene statt.
- Aus einem Exemplar der Klasse B (im selben Package wie A) sind neben öffentlichen Methode und solchen mit Default-Sichtbarkeit auch die mit der Sichtbarkeit mit der Sichtbarkeit protected sichtbar. Wie man also sieht: Die Sichtbarkeitsstufe protected schließt die Default-Sichtbarkeit (gleiche Package) ein (das liest man immer wieder anders).
- Aus einem Exemplar der Klasse C sind nur die öffentlichen Methoden des A-Exemplars sichtbar. Das zeigt, dass das Package a.a *kein* Teil des Packages a ist, d.h. die Packages bilden in Java *keine* echte Hierarchie.
- Aus einem Exemplar der Klasse D (welche Subklasse von A ist) sind die als protected deklarierten Methoden eines A-Exemplars *nicht* sichtbar, wenn die Referenz, über die zugegriffen wird, den statischen Typ A hat. Hat sie hingegen den statischen Typ D, *sind* sie sichtbar, obwohl ja ein D-Exemplar auch ein A-Exemplar ist. Diese Spezialität der Sichtbarkeitsstufe protected ist teilweise auch langjährigen Java-Programmierern nicht bekannt.

Package bsp_11_awt

Dieses Beispiel ist ein minimales AWT-Beispiel: Ein Fenster mit einem Panel und einem Button. Einige Aspekte sind hier erwähnenswert:

1. In der main()-Methode wird lediglich der Konstruktor aufgerufen, sonst passiert dort nichts. Dennoch ist das Programm nach der Ausführung von main() offenbar nicht zu Ende, sondern läuft weiter. Daraus kann man schließen, dass mit der Erzeugung und Sichtbarmachung des Fensters implizit mindestens ein weiterer Thread neben dem Main-Thread gestartet wird, der das Ende des Main-Threads überdauert. Das ist tatsächlich der

Fall, es handelt sich um eine ganze Gruppe von Threads, die sich um die Ereignisverarbeitung des AWT kümmern. Wenn das Fenster erzeugt ist, wartet ein VM-Thread auf Ereignisse, die vom Betriebssystem an die VM gemeldet werden. Die VM analysiert, an welcher Komponente ein Ereignis aufgetreten ist und stellt es in eine Ereigniswarteschlange (Queue) ein. Ein weiterer Thread, der sogenannte Event-Dispatch-Thread (EDT) schaut immer wieder in diese Queue und wenn er dort ein Ereignis findet, wird es abgearbeitet, indem der EDT die ereignisverarbeitenden Methoden der bei der Ereignisquelle für dieses Ereignis angemeldeten Listener ausführt.

2. In Zeile 20 erzeugen wir ein Panel-Exemplar. Zeile 21 zeigt, dass auch aus einem Konstruktor bereits Methoden des gerade in Initialisierung begriffenen Objekts aufgerufen werden können. Und in Zeile 22 wird das Panel mit Hilfe der von Frame geerbten Methode `add()` unserem `MyFrame`-Exemplar hinzugefügt. Ein `Frame` ist Subtyp von `Container`. `Container` sind all jene AWT-Komponenten, die als „Behälter“ für andere Komponenten dienen können. „Behälter“ ist hier bezogen auf die Oberfläche, also nicht im Sinne von `Container`-Klassen wie `LinkedList` oder `ArrayList`.

3. In Zeile 24 wird ein `Button` erzeugt, der in Zeile 30 ebenfalls mit `add()` dem `Container`s Oberflächenelement hinzugefügt wird. Diesmal wird `add()` allerdings nicht nur die Komponente (der `Button`) übergeben, sondern ein weiterer Wert. Offensichtlich ist die Methode `add()` der Klasse `Frame` überladen, d.h. sie existiert mehrmals mit unterschiedlichen Parametertypen. Bei dem zweiten Wert handelt es sich um ein statisches Attribut der Klasse `BorderLayout`, welches angibt, wo der `Button` erscheinen soll. Das AWT verwendet sog. `LayoutManager`, welche den `Containern` zugeordnet werden und für die Anordnung der Elemente des `Containers` sorgen. Ein `Frame` hat als Vorgabe ein `BorderLayout` und der übergebene Parameter sorgt dafür, dass der `Button` im Bereich `SOUTH` des `Frames` erscheinen wird. Das `Panel` hingegen wurde in den Bereich `CENTER` gelegt werden: Wenn bei `add()` kein Bereich explizit angegeben wird, legt ein `BorderLayout` die Komponente ins `CENTER`.

4. In den Zeilen 24-29 wird bei dem `Button` ein `Listener` angemeldet. Die Methode `addActionListener()` erwartet als Parameter ein Objekt, welche das Interface `ActionListener` implementiert. Statt nun eine eigene Klasse als Subtyp von `ActionListener` zu implementieren, wird hier eine sog. lokale anonyme innere Klasse verwendet: Wir erzeugen ein Exemplar einer Klasse, über welche nur bekannt ist, dass sie das Interface `ActionListener` implementiert, wobei die Methode `actionPerformed()` so implementiert wird, wie in den Zeilen 25-28 beschrieben. Die Klasse wird also ad hoc deklariert, ohne dass sie einen Klassennamen bekommt und es wird direkt eine Instanz erzeugt und verwendet. Das gleiche Verfahren wird später für den `WindowListener` angewendet, wobei dort die anonyme Klasse von `WindowAdapter` (eine Implementierung von `WindowListener` mit leeren Methodenrümpfen) abgeleitet wird und nur die eine Methode überschreibt, für die wir uns im `Listener` tatsächlich interessieren.

Klasse Test_1:

In Zeile 7 wird eine Methode aufgerufen, welche deklariert, dass sie eine `OutOfBoundsException` auslösen kann. Da eine `OutOfBoundsException` eine `CheckedException` (Subklasse von `Exception`, aber nicht von `RuntimeException`) ist, muss der Methodenaufruf entweder in einen `try/catch`-Block mit passender `Catch`-Klausel gekapselt wird oder die Methode, in der der Aufruf steht, muss selbst deklarieren, die betreffende `Exception` auslösen zu können. Keines von beidem ist hier gegeben → Compilerfehler. Sehen Sie sich an, welche `QuickfixOptionen` Eclipse bietet, um den Compilerfehler zu beseitigen.

Im darauffolgenden Codeblock wurde der Aufruf nun in `try/catch` gekapselt. Es gibt *mehrere* `Catch`-Blöcke, von denen beim Abfangen einer `Exception` immer der erste passende gewählt wird. Außerdem existiert ein `Finally`-Block. Dieser wird unabhängig davon *immer* ausgeführt, ob der dazugehörige `try`-Block abrupt terminierte oder normal beendet wurde.

In der Methode `aMethod()` sehen wir, wie eine `Exception` ggf. tatsächlich ausgelöst wird, nämlich mit dem Schlüsselwort `throw`, gefolgt von einem Ausdruck, der ein `Exception`-Objekt referenziert, typischerweise ist dies der Aufruf eines Konstruktors, der ein solches Objekt erzeugt.

Die Klasse `OutOfBoundsException` schließlich zeigt, dass eine `Exception`-Klasse eine ganz normale Klasse ist, deren Exemplare Attribute und Methoden haben können. Das Attribut `value` wird hier verwendet, um beim Auslösen der `Exception` die Information weiterzuleiten, *welcher* Wert eigentlich zur Auslösung geführt hat.

Klasse Test_2:

Hier haben wir in der Methode `m1()` einen `catch`-Block, der den Ausnahmetyp `OutOfBoundsException` behandeln soll. Die Klasse `Out-OfBoundsException` ist Subklasse von `Exception`, aber nicht von `RuntimeException`, es handelt sich also um eine `CheckedException`. Da der Compiler erkennt, dass diese `Exception` im betreffenden `try`-Block nicht ausgelöst werden *kann* – im `try`-Block befindet sich ja kein Aufruf einer Methode, die deklariert, eine solche `Exception` auslösen zu können – meldet er einen Fehler "Unreachable catch block for `OutOfBoundsException`. This exception is never thrown from the try statement body" (Eclipse Compiler).

In der Methode `m2()` haben wir einen `try`-Block mit zwei `catch`-Blöcken, von denen der erste den Ausnahmetyp `Exception`, der zweite den Ausnahmetyp `OutOfBoundsException` behandeln soll. Da aber bei Auftreten einer `OutOfBoundsException` bereits der erste `catchBlock` "passt" (`OutOfBoundsException` ist ja Subtyp von `Exception`), kann der zweite `catchBlock` nie zur Ausführung kommen. Dies merkt der Compiler und meldet einen Fehler "Unreachable catch block for `OutOfBoundsException`. It is already handled by the catch block for `Exception`" (Eclipse Compiler).

Die Klasse Test1 zeigt einige grundlegende Eigenschaften von Enums:

Die Enum Wochentag repräsentiert Wochentage, d.h. jedes Exemplar dieser Klasse steht für einen Wochentag. Wie man sieht, kann eine Enum neben der Aufzählung ihrer Werte auch Methoden enthalten. *Jede* Enum verfügt über die Methoden toString() und ordinal(). Letztere liefert die Stelle, an der der betreffende Enum-Wert in der Aufzählung steht (beginnend mit 0). Außerdem verfügt eine Enum-Klasse immer über die statischen Methoden values(), welche die für diese Enum-Klasse definierten Werte als Array liefert und valueOf(String name), welche zu einem String das entsprechend benannte Exemplar der Enum liefert, falls ein solches existiert. Ist für die Enum kein Wert definiert, der dem übergebenen String entspricht, löst valueOf(String name) eine IllegalArgumentException aus.

Die Klasse Test2 zeigt, dass eine Enum auch Attribute haben kann. Das sind typischerweise Konstanten (final), die für die einzelnen Enum-Werte unterschiedlich belegt sind. Ihr Hauptzweck ist es, den Enum-Werten weitere unveränderliche Eigenschaften fest zuzuordnen, hier jedem Planet die Anzahl seiner Monde. Wie man sieht, wird die Konstante numberOfMoons in einem Konstruktor initialisiert. Dieser Konstruktor ist als private deklariert. Das wird zwar vom Compiler nicht erzwungen, ist aber sinnvoll, weil eine Erzeugung weiterer Enum-Werte über die in der Klasse aufgezählten hinaus ohnehin nicht möglich ist. Benutzt wird der Konstruktor von der VM (die darf das): Diese erzeugt die den Enum-Werten entsprechenden Objekte, wenn die Enum-Klasse das erste mal verwendet wird und nutzt dazu die Aufzählung der Werte selbst (Zeile 13) als Folge von Konstruktoraufrufen. Führt man den Test aus, sieht man durch die Ausgabe-Anweisung in Zeile 18 die Erzeugung und Initialisierung der Objekte durch die VM.

Das erste Beispiel (Konto1.java) zeigt Code, der in einer Multithreading-Umgebung zu Daten-Inkonsistenzen führen kann. Problemszenario: Thread 1 betritt die Methode einzahlen() und liest, um den rechten Teil der Zuweisung auszuwerten, den Wert von „saldo“ und addiert den „betrag“ hinzu. Nun unterbricht der Scheduler diesen Thread und Thread 2 wird rechnend. Auch dieser betritt die Methode einzahlen(), liest den saldo, addiert einen betrag auf und schreibt den neuen saldo zurück. Nun wird wieder Thread 1 rechnend und schreibt den neuen saldo zurück, den er vorhin errechnet hat. Ergebnis: Die von Thread 2 vorgenommene Einzahlung ist verloren gegangen.

Das zweite Beispiel (Konto2.java) zeigt Code, der in einer Multithreading-Umgebung zu einem Deadlock führen kann. Problemszenario: Thread 1 möchte einen Betrag von Konto A auf Konto B umbuchen, Thread 2 einen Betrag von Konto B auf Konto A. Thread 1 betritt die Methode umbuchenAuf() von Konto A und sperrt damit den Monitor von „this“, das ist hier Konto A. Nun wird Thread 2 rechnend und betritt die Methode umbuchenAuf() von Konto B und sperrt damit den Monitor von „this“, das ist hier Konto B. um weiterarbeiten zu können, müsste nun Thread 1 den auf „ziel“ synchronisierten Block betreten, also den Monitor von Konto B sperren, denn das ist ja für ihn „ziel“.. Diesen hat aber bereits Thread 2 gesperrt. Also kann Thread 1 erst weiterarbeiten, wenn Thread 2 diesen Monitor freigibt. Das wird aber nie geschehen, denn um weiterarbeiten zu können, müsste Thread 2 den auf „ziel“ synchronisierten Block betreten, also den Monitor von Konto A sperren, denn das ist für ihn „ziel“. Somit wartet jeder der beiden Threads auf den anderen → Deadlock. Sehen Sie sich bitte auch die Beispiele 14a und 14b an. Dort wird das Kontobeispiel einmal lauffähig ausprogrammiert und gezeigt, wie man sich den Deadlock in Eclipse „live“ ansehen kann.

Das dritte Beispiel (RunnableVsThread.java) zeigt die beiden Möglichkeiten, einen Thread in Java zu modellieren: Die Klasse Worker1 ist von der Klasse Thread abgeleitet und überschreibt deren Methode run(). Da Worker1 von Thread die Methode start() erbt, kann auf Exemplaren von Worker1 diese Methode direkt aufgerufen werden. Die Klasse Worker2 hingegen *hat* bereits eine Superklasse, kann also nicht auch noch von der Klasse Thread abgeleitet werden. Die Alternative besteht darin, sie vom Interfacetyp Runnable abzuleiten. Ein Runnable hat selbst keine Methode start(), es benötigt also ein Thread-Exemplar, damit seine run()-Methode in einem neuen Betriebssystem-Thread gestartet werden kann. Normalerweise wird das Runnable dazu einem Thread-Exemplar in dessen Konstruktor mitgegeben.

Das vierte Beispiel (Test.java) demonstriert, warum es sinnvoll ist, dass ereignisverarbeitende Methoden eines Listeners schnell zurückkehren und wie man dies erreicht, indem man länger dauernde Aktivitäten in einen eigenen Thread auslagert.

In Zeile 14 wird aus einem Listener heraus eine langwierige Aktion gestartet: Während diese ausgeführt wird, ist der AWT-Event Dispatch Thread blockiert, d.h. Ereignisse die in dieser Zeit am GUI auftreten, werden erst ausgeführt, nachdem die langwierige Aktion beendet ist. Probieren Sie es aus, indem Sie das Fenster vergrößern oder verkleinern (es erfolgt keine Layout-Neuberechnung) oder zu schließen versuchen. Kommentieren Sie hingegen die Zeile 14 aus und entkommentieren Zeile 15, wird die langwierige Berechnung

in einem neuen Thread gestartet und das GUI bleibt reaktiv. Vergleichen Sie bitte das Verhalten...

Die Zeilen 27-31 demonstrieren das Erzeugen eines neuen Threads, dem ein Runnable übergeben wird. Dieses Runnable ist hier ein Exemplar einer anonymen blocklokalen Klasse, die Runnable implementiert. Zu anonymen lokalen Klassen siehe auch den entsprechenden Abschnitt in bsp_11_awt.

Die Zeilen 33 und 34 demonstrieren, dass ein neuer Betriebssystem-Thread erst durch Aufruf von start() auf einem Thread-Exemplar gestartet wird: Wenn Sie Zeile 33 aus- und Zeile 34 entkommentieren, wird *kein* neuer Thread gestartet, d.h. die langwierige Aktion findet wieder im EDT statt.

Zeile 32 setzt die Priorität für das Thread-Exemplar auf den Standardwert. Dies ist sinnvoll, weil ein neu erzeugter Thread immer die Priorität des Threads erbt, aus dem er erzeugt wurde. Das wäre in diesem Fall der EDT, der eine relativ hohe Priorität hat, was für den neuen Thread i.A. nicht erwünscht ist.

Sehen Sie sich zum Thema "Threads" bitte auch Abschnitt 15 in der FAQ an, insbesondere die Erläuterungen zur Bedeutung von "synchronized".

Packages bsp_14a_Verklemmung_Problem und bsp_14b_Verklemmung_Loesung

Es werden zwei Konto-Instanzen mit einem Startwert von 1000 erzeugt, dann zwei Umbuchungs-Threads. Der eine bucht 100 vom ersten Konto aufs zweite, der andere 1100 vom zweiten Konto aufs erste. Dazu wird die Methode umbuchenAuf() des jeweiligen "Quellkontos" verwendet.

Um das Verlorengehen von Umbuchungen zu vermeiden, wird in dieser Methode sowohl das Quell- als auch das Zielkonto zur Synchronisation verwendet, d.h. ein Thread, der umbuchen möchte, muss die Monitore beider Konten sperren.

Beide Threads werden gestartet, anschließend wartet der Main-Thread per Aufruf von join() auf beiden, bis diese ihren Job getan haben und gibt anschließend die Kontostände aus. Die Ausgabe sollte sein:

Kontostand k1: 2000

Kontostand k2: 0

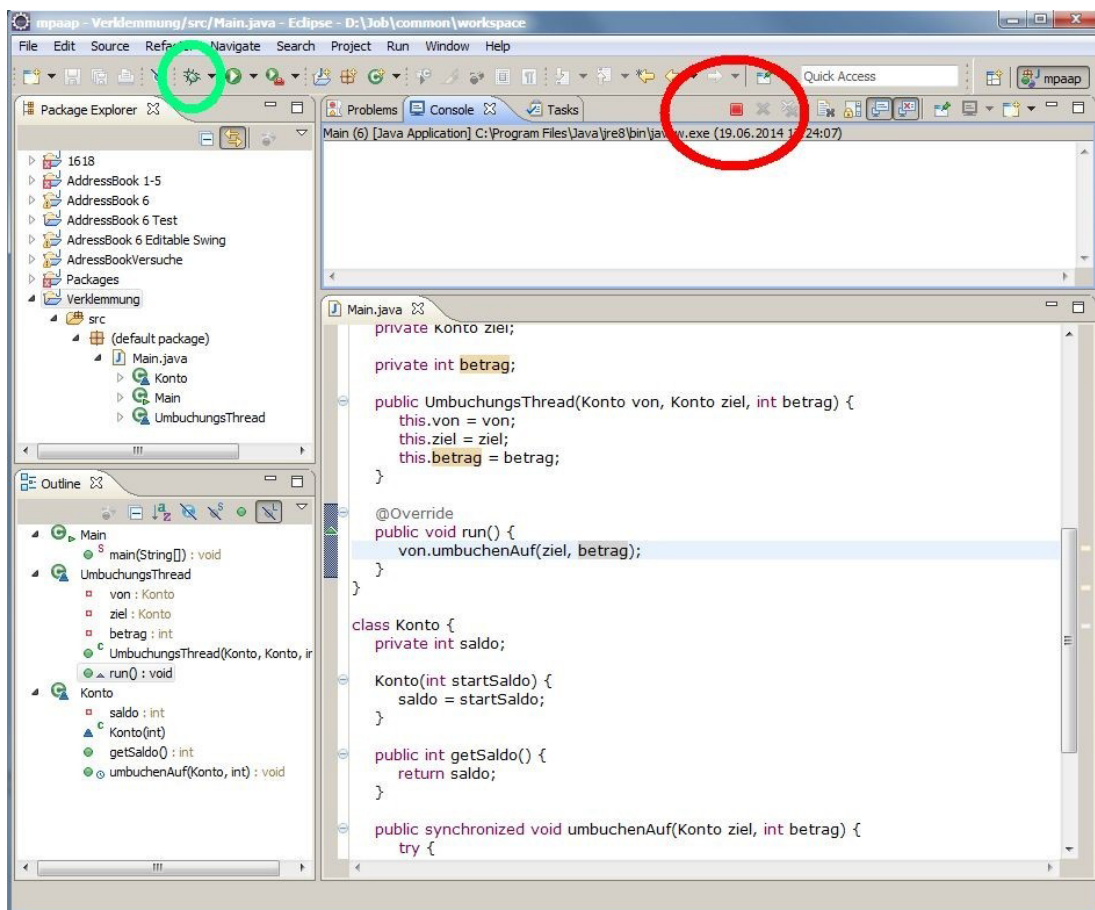
Das ist auch das, was wir normalerweise beobachten werden. Nun wissen wir aber, dass es aufgrund der Tatsache, dass beide Threads die Monitore beider Konten sperren wollen, und zwar wechselweise jeweils das andere zuerst, zu einer Verklemmung kommen kann, nämlich genau dann, wenn ein Thread den Monitor des Quellkontos seiner Aktion sperrt, dann durch den Scheduler unterbrochen wird und nun der andere Thread den Monitor des Quellkontos seiner Aktion sperrt, welches ja das Zielkonto des ersten Threads ist. Beide Threads warten jetzt darauf, den Monitor ihres Zielkontos sperren zu können, was nie der Fall sein wird.

Wie können wir dieses Szenario nun mutwillig heraufbeschwören? Ganz einfach... wir müssen ja nur die Wahrscheinlichkeit erhöhen, dass der Scheduler an genau der o.g. Stelle den Thread unterbricht und den anderen rechnen lässt. Dazu legen wir den Thread, der

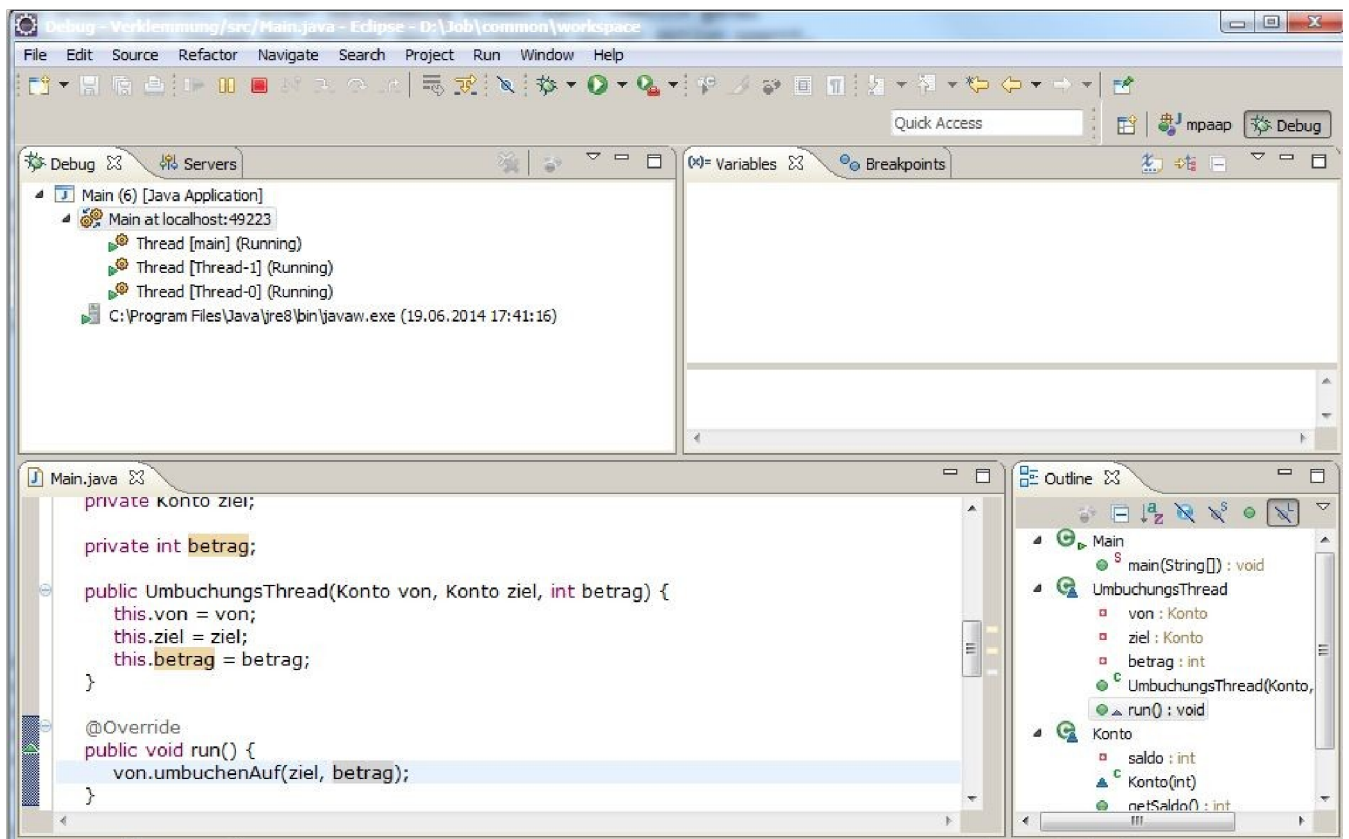
gerade den Monitor "seines" Quellkontos gesperrt hat, einfach für einen Moment schlafen, indem wir die Methode umbuchenAuf() wie folgt ändern:

```
public synchronized void umbuchenAuf(Konto ziel, int betrag) {  
    try {  
        Thread.sleep(50);  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
    synchronized (ziel) {  
        this.saldo =  
        this.saldo - betrag;    ziel.saldo =  
        ziel.saldo + betrag;    }  
}
```

Ausdrücklicher Hinweis: Diese Änderung bewirkt nichts, was nicht auch ohne die Sleep-Anweisung hätte geschehen können. Nur passiert es jetzt quasi mit Sicherheit, während es vorher unwahrscheinlich war. Wenn wir das Programm jetzt laufen lassen, bekommen wir keine Ausgabe. In Eclipse können Sie aber an dem roten Quadrat neben dem Console-View sehen, dass das im Bild Programm noch läuft:

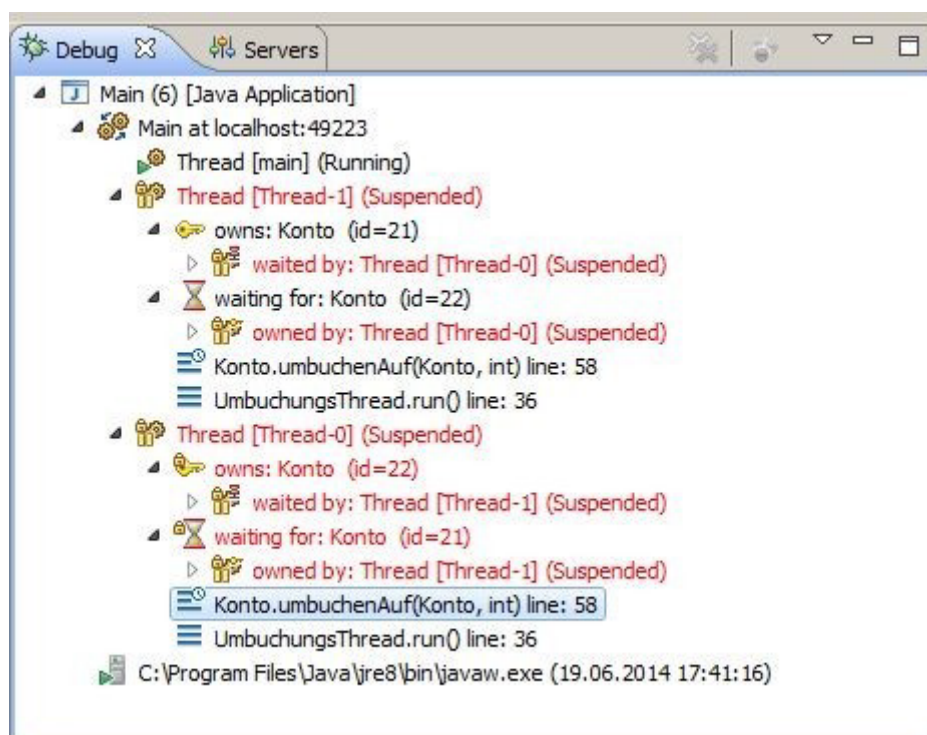


Brechen Sie zunächst die Ausführung durch Klick auf das rote Quadrat ab. Starten Sie das Programm dann erneut, aber diesmal nicht mit dem normalen "Run-Knopf", sondern mit dem "Debug-Knopf", den ich im Bild grün eingekreist habe. Sie werden gefragt, ob Eclipse in die Debug-Perspektive wechseln soll, was Sie bestätigen. Sie sollten nun ungefähr das hier sehen:



Interessant ist dabei die Anzeige der laufenden Threads im Debug-View oben links. Dort sehen Sie drei laufende Threads, den Main-Thread (klar, der wartet aufgrund von "Join" auf die anderen beiden) und die beiden Umbuchungs-Threads.

Um sich anzusehen, was die beiden Umbuchungs-Threads eigentlich tun, müssen Sie sie aus Eclipse heraus pausieren. Dazu klicken Sie mit der rechten Maustaste auf einen Thread und wählen dann aus dem Kontextmenü "Suspend". Anschließend können Sie durch Anklicken der kleinen weißen Dreiecke die jeweiligen "Information-Bäume" aufklappen. Das Ergebnis sehen Sie hier (diesmal nur das Debug-View):



Thread-1 hält ("owns") den Monitor eines Konto-Objekts (bei mir Objekt Nr. 21). Auf Objekt 21 wird von Thread-0 gewartet ("waited by"). Thread-1 wartet darauf ("waiting for"), den Monitor eines weiteren Konto-Objekts (Objekt Nr. 22) sperren zu können, das aber von Thread-0 gesperrt ist ("owned by"). Und entsprechend für Thread-0, nur umgekehrt. QED.

An welcher Stelle im Code die beiden Threads genau stehen, sehen Sie übrigens auch, und zwar hinter den Symbolen mit den drei waagrechten blauen Strichen: Die nächste Anweisung, die beide Threads ausführen würden, wenn Sie denn könnten, wäre Zeile 58, in der Methode umbuchen() der Klasse Konto. Und aufgerufen wurde diese Methode aus Zeile 36, in der Methode run() der Klasse UmbuchungsThread.

Package bsp_15_ueberladung

Dieses Beispiel demonstriert mehrere im Zusammenhang mit der Auflösung von Überladung wichtige Sachverhalte.

- Aufruf A: Der Compiler kennt nur statische Typen. Deshalb interessiert ihn bzgl. der Variablen f auch nicht, ob diese vielleicht zur Laufzeit eine Forelle referenzieren wird. Der statische Typ von f ist Fisch. Und daher ist die Methodensignatur m(Vogel, Forelle) für diesen Aufruf *nicht* passend. Passend zum Aufruf ist nur die Methodensignatur m(Tier, Fisch), ein entsprechender Aufruf wird in den Bytecode geschrieben.
- Aufruf B: Der Compiler kennt nur statische Typen. Die Variable s2 hat ebenso wie s1 den statischen Typ Super. In den Bytecode wird also wie bei Aufruf A ein Aufruf von m(Tier, Fisch) geschrieben. Zur Laufzeit allerdings stellt die VM fest, dass s2 ein Exemplar der Klasse Sub referenziert, sucht dort nach einer Methode mit der Signatur m(Tier, Fisch) und findet die Methode "6" (welche in der Klasse Sub die Methode "1" überschreibt). Also wird Methode "6" ausgeführt.
- Aufruf C: Der Compiler kennt nur statische Typen. Die Variable s1 hat den statischen Typ Super. Deswegen finden bei der Auflösung der Überladung auch nur Methodensignaturen Berücksichtigung, die für den Typ Super deklariert sind. Die Signatur m(Vogel, Huhn) wird also **nicht** berücksichtigt, und wenn sie von den Parametertypen noch so gut passen würde. Passend zum Aufruf ist in Super nur die Methodensignatur m(Tier, Huhn), ein entsprechender Aufruf wird in den Bytecode geschrieben.
- Aufruf D: Der Compiler kennt nur statische Typen. Die Variable s2 hat ebenso wie s1 den statischen Typ Super. In den Bytecode wird also wie bei Aufruf C ein Aufruf der Methode mit der Signatur m(Tier, Huhn) geschrieben. Zur Laufzeit allerdings stellt die VM fest, dass s2 ein Exemplar der Klasse Sub referenziert, und sucht in der Klasse Sub nach einer Methode mit der vom Compiler gewählten Signatur. Die findet sie nicht, also sucht sie in der Superklasse von Sub, also in Super. Dort findet sie die Methode "3" mit der gesuchten Signatur und führt diese aus. Die tatsächlichen Typen der übergebenen Parameter spielen bei Java zur Laufzeit *keine* Rolle (die Auflösung von Überladung erfolgt nur durch den Compiler und ist dann abgeschlossen), also kommt auch zur Laufzeit die Methode "5" nicht zum Zuge.
- Aufruf E: Der Compiler kennt nur statische Typen. Die Variable s1 hat den statischen Typ Super. Deswegen finden bei der Auflösung der Überladung auch nur

Methodensignaturen Berücksichtigung, die für den Typ Super deklariert sind. Die statischen Typen der Parameter von Aufruf E sind Huhn|Forelle. Dazu passen die Methodensignaturen $m(\text{Tier}, \text{Fisch})$, denn $\text{Huhn SUB-} \rightarrow \text{Tier}$, $\text{Forelle SUB-} \rightarrow \text{Fisch}$ und $m(\text{Vogel}, \text{Forelle})$, denn $\text{Huhn SUB-} \rightarrow \text{Vogel}$, $\text{Forelle SUB-} \rightarrow \text{Forelle}$. Der Compiler muss also entscheiden, welche der passenden Methodensignaturen die speziellste ist. Das tut er, indem er aus der Liste der passenden Signaturen alle streicht, zu denen es eine speziellere gibt. Spezieller als eine Methodensignatur a ist eine Methodensignatur b genau dann, wenn für alle deklarierten Typen der formalen Parameter von b gilt, das ihr statischer Typ Subtyp des statischen Typs des entsprechenden a -Parameters ist. Die Frage, welche von zwei Methodensignaturen spezieller ist, lässt sich also unabhängig von einem konkreten Aufruf beantworten! In diesem Fall stellt der Compiler fest, dass die Methodensignatur $m(\text{Vogel}, \text{Forelle})$ spezieller als die Methodensignatur $m(\text{Tier}, \text{Fisch})$ ist, denn es gilt: $\text{Vogel SUB-} \rightarrow \text{Tier}$ und $\text{Forelle SUB-} \rightarrow \text{Fisch}$. Die Signatur $m(\text{Tier}, \text{Fisch})$ wird also aus der Liste gestrichen, damit ist nur noch $m(\text{Vogel}, \text{Forelle})$ übrig und damit die speziellste zum Aufruf passende Methodensignatur, also wird ein entsprechender Aufruf in den Bytecode geschrieben.

- Aufruf F: Der Compiler kennt nur statische Typen. Die Variable $s1$ hat den statischen Typ Super. Deswegen finden bei der Auflösung der Überladung auch nur Methodensignaturen Berücksichtigung, die für den Typ Super deklariert sind. Die statischen Typen der Parameter von Aufruf F sind Huhn|Huhn. Dazu passen die Methodensignatur $m(\text{Tier}, \text{Huhn})$, denn $\text{Huhn SUB-} \rightarrow \text{Tier}$, $\text{Huhn SUB-} \rightarrow \text{Huhn}$ und $m(\text{Huhn}, \text{Vogel})$, denn $\text{Huhn SUB-} \rightarrow \text{Huhn}$, $\text{Huhn SUB-} \rightarrow \text{Vogel}$. Der Compiler muss also entscheiden, welche der passenden Methodensignaturen die speziellste ist. Das tut er, indem er aus der Liste der passenden Methodensignaturen alle streicht, zu denen es eine speziellere gibt. Spezieller als eine Methodensignatur a ist eine Methodensignatur b genau dann, wenn für alle deklarierten Typen der Parameter von b gilt, das ihr statischer Typ Subtyp des statischen Typs des entsprechenden a -Parameters ist. Damit $m(\text{Tier}, \text{Huhn})$ spezieller wäre als $m(\text{Huhn}, \text{Vogel})$, müsste also gelten: $\text{Tier SUB-} \rightarrow \text{Huhn}$ und $\text{Huhn SUB-} \rightarrow \text{Vogel}$. Die erste Bedingung ist nicht erfüllt, also ist $m(\text{Tier}, \text{Huhn})$ *nicht* spezieller als $m(\text{Huhn}, \text{Vogel})$. Damit $m(\text{Huhn}, \text{Vogel})$ spezieller wäre als $m(\text{Tier}, \text{Huhn})$, müsste gelten: $\text{Huhn SUB-} \rightarrow \text{Tier}$ und $\text{Vogel SUB-} \rightarrow \text{Huhn}$. Die zweite Bedingung ist nicht erfüllt, also ist $m(\text{Huhn}, \text{Vogel})$ *nicht* spezieller wäre als $m(\text{Tier}, \text{Huhn})$. Damit kann keine der beiden Methodensignaturen gestrichen werden, es bleiben zwei gleich spezielle passende Methodensignaturen, somit ist der Aufruf F für den Compiler nicht auflösbar. → Compilerfehler.

Es empfiehlt sich, bei der Auflösung von Überladung stur nach dem vorgegebenen Algorithmus vorzugehen, also zunächst alle zum Aufruf passenden Methodensignaturen auszuwählen, wenn es mehrere sind, wirklich für jede passende Methodensignatur a zu schauen, ob es zu dieser eine speziellere ebenfalls passende Methodensignatur b gibt und wenn ja, a zu streichen. Nur so ist man vor fehlerhaften Schnellschlüssen sicher, wie z.B. dem, dass bzgl. des Aufrufs F die Methode "4" passender sei, weil deren Parametertypen Huhn|Vogel "näher" an Huhn|Huhn seien als die Parametertypen Tier|Huhn der Methode "3", oder dem, dass Methode "4" spezieller sei als Methode "3", weil Huhn|Vogel "weiter unten in der Typhierarchie liege" als Tier|Huhn. Das ist aber alles völlig uninteressant!

Sollten zum Thema „Auflösung von Methodenaufrufen“ noch Unklarheiten verblieben sein, lesen Sie bitte unbedingt Abschnitt 13 der 63611-FAQ!

Package bsp_16_arraystoreexception

Dieses Beispiel demonstriert Java-Code, der vom Compiler akzeptiert wird, aber zur Laufzeit eine `ArrayStoreException` auslöst. Ursache des Problems ist die Entscheidung Suns, aus einer bestehenden Subtypbeziehung zwischen Basistypen eine entsprechende Subtypbeziehung zwischen Arraytypen zu folgern, also `T[]` als Subtyp von `S[]` zu sehen, wenn `T` Subtyp von `S` ist.

Package bsp_17_wildcards

Dieses Paket enthält zwei Beispiele, `WildcardTest` und `WildcardSubtyping`.

Hinweis für Beleger von Modul 63611: Dass ein Typparameter oder eine Wildcard auch per „super“ beschränkt werden kann, wird im Modul 63611 nicht behandelt. Im Beispiel `Wildcard-Test` geht also der dritte Codeblock über den Kursstoff hinaus. Dasselbe gilt im Beispiel `WildcardSubtyping` für alle Zeilen, welche die Variable `listD` verwenden.

Die Klasse `WildcardTest` zeigt, was die Deklaration einer Variablen mit einem Wildcard-Typ für die Menge der potentiell von dieser Variablen referenzierten Objekte aussagt, also, welche Eigenschaften der Compiler bzgl. dieser Objekte als sicher annehmen kann und welche nicht, welche Operationen er also demzufolge auf der Variablen zulässt: und welche er verbietet.

(Nur zur Sicherheit sei angemerkt, dass die Tatsache, dass den Variablen `liste1`, `liste2` und `liste3` der Wert `null` zugewiesen wurde, absolut nichts mit den Fehlermeldungen des Compilers zu tun hat! Der Wert `null` (die leere Referenz) ist gültiger Wert *jeden* Typs und in den jeweils auf die Zuweisung folgenden Zeilen interessiert es den Compiler nicht im mindesten, was die Variable tatsächlich zur Laufzeit referenzieren wird, er kennt lediglich den statischen Typ und trifft seine Entscheidungen *ausschließlich* aufgrund dieses Typs. Manche IDEs werden Sie warnen, dass die Variablen mit `null` initialisiert wurden, das können Sie aber für das Thema, um das es uns hier geht, ignorieren.)

Durch die Beschränkung der Wildcard bei `liste2` und `liste3` wird letztlich eingeschränkt, welche Typparameter jemand bei der Instantiierung einer konkreten Liste verwenden darf, die einer dieser Variablen zugewiesen werden kann. Und daraus resultiert für den Compiler ein Wissen über die Objekte, welche in einer solchen Liste vorhanden sein können.

Der Variablen `liste1` können Listen mit beliebigem Parametertyp zugewiesen werden, d.h. dass der Compiler über diesen Typ nichts weiß. Es könnte sich also z.B. um eine `ArrayList<Tier>` handeln, aber auch um eine `ArrayList<Object>`. Also kann der Compiler uns nicht erlauben, über die Variable `liste1` *irgendetwas* in die Liste zu füllen. Und bzgl. eventuell aus der List zurückgegebener Objekte weiß er auch nicht mehr, als dass sie mit Sicherheit den Typ `Object` haben werden (denn den haben alle Objekte), weshalb er auch nur eine Zuweisung von aus der Liste geholten Objekten an eine Variable dieses Typs zulässt.

Der Variablen `liste2` können Listen mit allen Parametertypen zugewiesen, die Subtyp von `Tier` sind (wie immer schließt das `Tier` selbst ein). Der Compiler kann uns nach wie vor nicht erlauben, *irgendetwas* in die Liste zu füllen, auch kein `Tier`, denn die tatsächlich von `liste2` referenzierte Liste könnte auch z.B. eine `ArrayList<Vogel>` sein, und in die kann ich ja nicht jedes `Tier` packen. Über eventuell aus der Liste zurückgegebene Objekte weiß der Compiler allerdings, dass ihr Typ mit Sicherheit Subtyp von `Tier` sein wird, weswegen er eine Zuweisung von aus der Liste geholten Objekten an die Variablen zulässt, die einen Supertyp von `Tier` als statischen Typ haben.

Der Variablen `liste3` schließlich können Listen mit allen Parametertypen zugewiesen, die Supertyp von `Vogel` sind (wie immer schließt das `Vogel` selbst ein). Der Compiler kann uns nun erlauben, Objekte aller Subtypen von `Vogel` in die Liste zu füllen. Hingegen kann er über aus der Liste zurückgegebene Objekte mit Sicherheit nichts mehr sagen, außer, dass sie vom Typ `Object` sein werden: Im Extremfall könnte es sich tatsächlich z.B. um eine `ArrayList<Object>` handeln. Deswegen lässt er nur eine Zuweisung von aus der Liste geholten Objekten an Variablen des statischen Typs `Object` zu.

Die Klasse `WildcardSubtyping` beschäftigt sich mit der Frage, welche Subtypbeziehung zwischen verschiedenen Wildcardtypen besteht. Getestet wird das Vorliegen einer Subtypbeziehung hier anhand von Zuweisungen: Damit einem Ausdruck mit statischem Typ `A` ein anderer Ausdruck mit statischem Typ `B` zugewiesen werden kann, muss `B` zuweisungskompatibel zu `A` sein. Bei Referenztypen bedeutet das, dass `B` Subtyp von `A` sein muss.

Um dieses Beispiel zu verstehen, müssen Sie sich klar machen für welche Objektmenge die betreffenden Typen stehen: Damit der Compiler die Zuweisung erlaubt, muss sichergestellt sein, dass der Typ rechts vom Zuweisungsoperator in seiner Objektmenge keine Elemente hat, die nicht auch in der Objektmenge des Typs links vom Zuweisungsoperator vorkommen können. Schauen wir uns die Objektmengen an:

- Die Variable `listA` hat den statischen Typ `List<?>`. Dieser Typ steht für *alle* Listen, egal, mit welchem Typ diese parametrisiert wurden.
- Die Variable `listB` hat den statischen Typ `List<Object>`. Dieser Typ steht für Listen, die mit genau dem Typ `Object` parametrisiert wurden.
- Die Variable `listC` hat den statischen Typ `List<? extends Object>`. Dieser Typ steht für Listen, die mit einem Subtyp von `Object` parametrisiert wurden. Nun sind aber ja *alle* Referenztypen Subtypen von `Object`! Und damit beschreibt der Typ `List<? extends Object>` genau dieselbe Objektmenge wie der Typ `List<?>`.
- Die Variable `listD` hat den statischen Typ `List<? super Object>`. Dieser Typ steht für Listen, die mit einem Supertyp von `Object` beschränkt wurden. Nun gibt es aber nur genau einen Typ, der Supertyp von `Object` ist, nämlich der Typ `Object` selbst. Und damit beschreibt der Typ `List<? super Object>` genau dieselbe Objektmenge wie der Typ `List<Object>`.
- Die Variable `listE` hat den statischen Typ `List<String>`. Dieser Typ steht für Listen, die mit dem Typ `String` parametrisiert wurden.

Welche Zuweisungen nicht nun erlaubt?

- Der Variablen listA kann man jede der anderen vier Variablen zuweisen. Denn ihr statischem Typ umfasst *alle* Listen, egal, mit welchem Typ diese parametrisiert wurden.
- Der Variablen listB kann man von den anderen Variablen nur die Variable listD zuweisen, weil deren Typ die gleiche Objektmenge beschreibt wie der Typ von listB. Die anderen Variablen beschreiben Objektmengen, zu denen Listen gehören, die nicht mit *genau* dem Typ Object parametrisiert wurden.
- Für die Variable listC gilt dasselbe wie für listA, denn wie oben erklärt, beschreiben die statischen Ttypen von listA und listC dieselbe Objektmenge.
- Der Variablen listD kann man von den anderen Variablen nur die Variable listB zuweisen, weil deren Typ die gleiche Objektmenge beschreibt wie der Typ von listD. Die anderen Variablen beschreiben Objektmengen, zu denen Listen gehören, die nicht dem einzig existierenden Supertyp von Object, also Object, parametrisiert wurden.

Beachten Sie bitte insbesondere, dass die Zuweisung von listE an listB *nicht* erlaubt ist. Denn der Typ List<String> ist *nicht* Subtyp des Typs List<Object>, weil sich die vorhandene Subtypbeziehung zwischen den Typparametern nicht auf die parametrisierten Typen überträgt (weshalb die Wildcardtypen ja überhaupt erst eingeführt werden mussten). Denn eine List<Object> ist eine Liste, in die sie beliebige Objekte legen können und genau das ist eine List<String> eben nicht, denn in die können Sie nur Strings legen.

Package bsp_18_wildcards

Beim Umgang mit Wildcardtypen und dem Erleben der mit ihnen verbundenen Einschränkungen (siehe Package bsp_17_wildcards) kommt oft die Frage auf, was man denn mit diesen Typen überhaupt anfangen kann. Die Antwort lautet: Wildcardtypen werden verwendet, wenn es darum geht, mit Objektmengen typsicher zu hantieren, bei denen es auf den konkreten Typparameter nicht ankommt. Z.B. kann man Sortieralgorithmen implementieren, die alle Listen sortieren können, deren Elementtyp ein Subtyp von Comparable ist, weil diese Voraussetzung für die Arbeit des Algorithmus reicht. Die Menge all dieser Listen wird aber genau durch den Typ List<? extends Comparable> beschrieben. Die Beispielklasse ListenListen demonstriert etwas Ähnliches: Mit Hilfe von Wildcardtypen ist es möglich, verschiedene Listen mit jeweils festgelegtem Parametertyp gemeinsam zu behandeln, indem ein passender Supertyp, nämlich "Listen, die mit einem Subtyp von Zootier parametrisiert sind" verwendet wird.

Die ImportExportList-Klassen sind ein nettes Beispiel „aus der Praxis“, welches immer wieder gerne genommen wird, wenn man begründen möchte, wofür man eigentlich nach *unten* beschränkte Parametertypen (werden im Kurstext nicht erwähnt) benötigt. Es handelt sich um eine Liste, die eine Methode hat, welche alle Elemente einer übergebenen Liste als eigene Elemente übernimmt, und eine weitere Methode, welche alle eigenen Elemente in eine übergebene Liste füllt. Ich hoffe, es ist selbsterklärend, warum die Typen der Parameter der beiden Methoden genau so beschränkt sein müssen, wie sie es sind.

Package bsp_19_fragile_base_class

Objekte der Klasse `IntegerBehaelter` haben eine Methode `add()`, mit der man ein `Integer` Exemplar in den Behälter legen kann. Außerdem gibt es eine Methode `addAll()`, der man eine `Collection<Integer>` übergeben kann und die *alle* `Integer`s der `Collection` in den Behälter legt. Diese Methode `addAll()` ist so implementiert, dass sie über die übergebene `Collection` iteriert und für jedes `Integer` die Methode `add()` des Behälters verwendet, um es in den Behälter zu legen („erste Implementierung“).

Nun hat ein anderer Programmierer eine neue Klasse von `IntegerBehaelter` abgeleitet. Objekte dieser Klasse `AddierenderIntegerBehaelter` aktualisieren bei jedem Hineinlegen eines `Integer`s ein Attribut, welches die Summe aller `Integer`s im Behälter enthält. Dazu hat der Programmierer von `AddierenderIntegerBehaelter` die Methode `add()` aus `IntegerBehaelter` entsprechend überschrieben. Und da auch die (geerbte) Methode `addAll()` bei einem `AddierenderIntegerBehaelter` per dynamischer Methodenwahl intern die *neue* `add()`-Methode verwendet, wird auch beim Aufruf von `addAll()` auf einem Exemplar von `AddierenderIntegerBehaelter` die Summe korrekt aktualisiert.

Wenn nun aber der Programmierer von `IntegerBehaelter` irgendwann beschließt, statt in `addAll()` selbst über die `Collection` zu iterieren, dies lieber direkt die interne `ArrayList`, in der die `Integer`-Werte gelagert sind, erledigen zu lassen – diese verfügt über eine entsprechende Methode – dann führt diese scheinbar harmlose Änderung („zweite Implementierung“) in der Superklasse `IntegerBehaelter` dazu, dass die abgeleitete Klasse `AddierenderIntegerBehaelter` nicht mehr korrekt arbeitet, sondern eine falsche Summe liefert. Denn nun verwendet `addAll()` ja `add()` nicht mehr,

Package bsp_20_quadrat_rechteck_mit_offener_rekursion

In der Übersetzungseinheit `Rechteck1.java` wurde aus Gründen maximaler Code-Wiederverwendung `Rechteck` von `Quadrat` abgeleitet. Der Preis, den man dafür zahlt, ist hoch: Die abgeleitete Klasse `Rechteck` ist in Wirklichkeit überhaupt keine Spezialisierung von `Quadrat`, denn die Menge der Quadrate ist eine Teilmenge der Menge der Rechtecke und nicht umgekehrt!

In `Rechteck2.java` wurde der umgekehrte Weg beschritten: `Quadrat` wurde von `Rechteck` abgeleitet, was unter dem Gesichtspunkt von Generalisierung/Spezialisierung korrekt ist. Diese Variante hat aber den Nachteil, dass zum einen `Quadrat` jetzt ein im Grunde überflüssiges Attribut hat, und zum anderen ein `Quadrat` nur dadurch ein `Quadrat` ist, dass man implizit eine neue Klasseninvariante eingeführt hat, nämlich, dass `laenge` und `breite` immer denselben Wert haben müssen. Um deren Einhaltung unter allen denkbaren Umständen muss sich der Programmierer ab jetzt kümmern. Das ist bei diesem einfachen Beispiel vielleicht noch vertretbar, bei einem komplexeren Szenario kann das aber durchaus problematisch werden.

`Rechteck3.java` schließlich zeigt eine elegante Lösung, welche mit offener Rekursion arbeitet. Sowohl `Quadrate` als auch nichtquadratische Rechtecke sind von einer gemeinsamen abstrakten Superklasse abgeleitet, in der die beiden Methoden `flaeche()` und `umfang()` bereits voll ausprogrammiert sind, aber intern zwei neue abstrakte Methoden `laenge()` und `breite()` verwenden, die korrekt zu implementieren, den Subklassen auferlegt

wird. Diese tun dies auf unterschiedliche Weise. Diese Lösung vermeidet die Nachteile der beiden ersten Varianten. Und als zusätzliches „Bonbon“ hält sie sich auch noch an die Regel, dass nach Möglichkeit nur von den „Blättern“ des Klassenbaums Exemplare existieren sollten, während Superklassen möglichst abstrakt sein sollten.

Package bsp_21_listener

Am Beispiel eines Weckdienstes wird die Funktionsweise von Listnern gezeigt.

Es geht darum, einer Ereignisquelle mitzuteilen, was sie beim Eintreten „ihres“ Ereignisses tun soll. Dies passiert, indem man bei der Ereignisquelle Interessenten anmeldet. Tritt dann später das Ereignis ein, werden diese Interessenten per Aufruf einer ganz bestimmten Methode benachrichtigt („Callback“). Um sicherzustellen, dass diese Methode auch vorhanden ist, dürfen nur solche Objekte als Interessenten angemeldet werden, die von einem bestimmten Typ sind, nämlich eben von dem Typ, der die Existenz der Methode beschreibt.

Im Beispiel ist die Ereignisquelle ein Weckdienst. Dieser hat eine Methode, mit der man WeckdienstKunden beim Weckdienst anmelden kann. WeckdienstKunde ist ein Interface, welches genau eine Methode deklariert, nämlich die Methode wecke(). Ein WeckdienstKunde ist also jemand, dem man die Nachricht wecke() senden kann. Und genau das tut der Weckdienst im Beispiel, wenn es sechs Uhr ist. Dabei übergibt er als Parameter ein Objekt, das zusätzliche Informationen enthält., nämlich eine Referenz auf den Weckdienst selbst und einen String mit der Uhrzeit. Diese Informationen stehen dann in der Methode wecke() des WeckdienstKunden zur Verfügung.

In der main()-Methode der Klasse Test werden bei einem Weckdienst zwei Kunden angemeldet, nämlich einmal ein Exemplar einer konkreten Klasse HotelGast und zum anderen ein Exemplar einer anonymen lokalen Klasse. Beide Klassen implementieren natürlich das Interface WeckdienstKunde.

Eine Abbildung dieses Beispiels auf ein analoges Konstrukt des AWT könnte so aussehen:

Weckdienst (Ereignisquelle)	ein Button (Ereignisquelle)
Methode anmelden() des Weckdienstes	die Methode addActionListener() des Buttons
Interface WeckdienstKunde	das Interface ActionListener
Methode wecke() in WeckdienstKunde	Methode actionPerformed() in ActionListener
WeckEreignis als Parameter von wecke()	ActionEvent als Parameter von actionPerformed()

Package bsp_22_inner_outer

Bitte beachten Sie: Dieses Beispiel zeigt keine auch nur im Entferntesten sinnvolle Verwendung innerer Klassen, sondern demonstriert lediglich bestimmte Gegebenheiten, die bei deren Verwendung beachtet werden müssen. Sinnvolle Beispiele finden Sie im Kurs im Abschnitt 2.2.2.1 (Entry und ListIterator im Kontext der Klasse LinkedList1618).

Ein Exemplar einer *nichtstatischen* inneren Klasse ist stets einem Exemplar der umgebenden Klasse zugeordnet. Auf dieses hat es eine implizite Referenz, über welche es auf die Methoden und Attribute des „umgebenden Exemplars“ zugreifen kann. Dies gilt

auch für private Methoden und Attribute. Ein solcher Zugriff wird im Beispiel in der Methode `m()` des Inner gezeigt (Zugriff auf `x`).

Außerdem sehen Sie dort den Fall, dass ein Attribut `y` des Exemplars der nichtstatischen inneren Klasse ein gleichnamiges Attribut `y` „seines“ Exemplars der umgebenden Klasse verdeckt und wie Sie dieses trotzdem ansprechen können.

In der `main()`-Methode der Klasse `Test` sehen Sie die spezielle Syntax, mit der Sie einem neuen Inner „sein“ Outer mitteilen können. Der bei weitem üblichere Fall ist aber, dass Inner-Exemplare aus einem Outer-Exemplar heraus erzeugt werden, welches dann automatisch das Outer wird, dem diese Inner-Exemplare zugeordnet sind.

Ein Exemplar einer *statischen* inneren Klasse hingegen ist keinem Exemplar der umgebenden Klasse zugeordnet und hat daher natürlich auch keine implizite Referenz auf ein solches. → Compilerfehler

Außerdem enthält das Package ein Beispiel, das zeigt, wie man mit Hilfe von Interfaces und inneren Klassen die tatsächliche Klasse eines Objekts vor Dritten komplett verbergen kann. Erläuterung als Kommentar direkt im Quellcode.

Package `bsp_23_objektgeflecht`

Zu Übungszwecken etwas komplizierter als die typische Klausur-Aufgabe. Welche der folgenden Aussagen bzgl. des Beispielcodes ist korrekt?

- a) Der Code ist nicht kompilierbar; es kommt zu einem Compilerfehler. Falls Sie sich für diese Aussage entschieden haben, nennen Sie bitte die Zeile, für welche der Compiler einen Fehler meldet.
- b) Der Code ist kompilierbar. Bei der Ausführung der `main()`-Methode der Klasse `Main` kommt es zu einem Laufzeitfehler. Falls Sie sich für diese Aussage entschieden haben, nennen Sie bitte den Laufzeitfehler, die Zeile, in welcher er auftritt und beschreiben Sie, warum er auftritt. Zeichnen Sie das Objektgeflecht, welches unmittelbar vor Ausführung dieser Zeile entstanden ist.
- c) Der Code ist kompilierbar. Bei der Ausführung der `main()`-Methode der Klasse `Main` tritt kein Laufzeitfehler auf. Falls Sie sich für diese Aussage entschieden haben, zeichnen Sie bitte das Objektgeflecht, das unmittelbar nach Ausführung der letzten Zeile der `main()`-Methode der Klasse `Main` entstanden ist.

Außerdem ist hier noch einmal der (leicht abgewandelte) Beispielcode aus Abschnitt 10.3.1 im Kurstext enthalten. Dort wird mit wenigen Zeilen ein Objektgeflecht aufgebaut, welches das Prinzip zeigt, nach dem die Stromklassen arbeiten.